DAIFEND
SECURING THE AI RUNTIME

TECHNICAL WHITEPAPER

DETECTION • POLICY • RECOVERY FOR ENTERPRISE AI

Daifend: AI Security Infrastructure for Agents, Vector Memory, and RAG

This paper describes Daifend's AI-native security model: how Daifend measures integrity, detects manipulation, enforces runtime policies, and restores trusted state for enterprise AI systems.

ABSTRACT

Modern AI systems are autonomous, tool-using, and stateful. They introduce new security domains: semantic manipulation, memory poisoning, retrieval chain corruption, and policy evasion in reasoning. Daifend provides a unified control plane for detection + enforcement + recovery.

DESIGN GOALS

- **Believable telemetry** that captures AI-native signals (drift, lineage, tool misuse, signatures).
- **Deterministic policy** for memory writes, retrieval, and tool execution.
- **Autonomous recovery** with auditable rollbacks and validation gates.
- **Enterprise integration** with SOC exports without becoming a generic SIEM.

1) Threat model for AI systems

THREAT SURFACES

Surface	Primary attacks	Impact	Daifend controls
MEMORY	Embedding poisoning, lineage tampering, semantic drift	Long-term manipulation, stealth corruption, persistent backdoors	Trust scoring, lineage enforcement, quarantines, rollback
RAG	Prompt injection, malicious docs, retriever bypass	Context hijack, unsafe actions, policy circumvention	Ingestion scanning, chain validation, signed context windows
AGENT	Tool misuse, permission escalation, stealth pivots	Data exfiltration, lateral movement, destructive actions	Sandboxing, policy gates, audit trails, session rotation
MODEL	Spec inversion, instruction hierarchy manipulation	Reliability collapse, unsafe compliance, covert behaviors	Signature correlation, policy constraints, response playbooks
IDENTITY	Synthetic persona, session spoofing	Unauthorized actions, trust abuse, stealth access	Behavior fingerprints, risk scoring, isolation

KEY OBSERVATION

AI attacks target meaning and state. Effective defense requires a runtime control plane that can: (1) observe semantic integrity, (2) enforce policy deterministically, and (3) restore trusted state autonomously.

2) Telemetry and integrity metrics

MEMORY TRUST SCORE (0–100)

A composited integrity score computed per memory namespace and time window.

- **Lineage confidence**: provenance completeness (ingestion → embed → write).
- **Semantic consistency**: drift relative to baseline embeddings.
- **Poison signatures**: known payload patterns in embedding space.
- **Write anomaly**: unusual burst writes, rare sources, unauthorized agents.

DRIFT SCORE (0–1)

Measures stability of meaning over time. Drift correlates with stealth poisoning and context steering.

```
drift = f(
  centroid_shift,
  neighborhood_entropy,
  retrieval_disagreement,
  lineage_break_rate
)
```

RUNTIME SIGNALS

- **Unsafe tool attempts**: blocked tool calls, restricted command patterns, anomalous parameters.
- **Containment actions**: quarantines, session rotations, rollback orchestration steps.
- **Threat signatures**: RAG bypass, embedding poisoning, model manipulation, synthetic identity.

3) Policy enforcement model

POLICY PRIMITIVES

- **Memory write policy**: required lineage, allowed sources, trust threshold, quarantine rules.
- **Retrieval policy**: signed context windows, maximum contamination budget, injection filters.
- **Tool policy**: risk-tiered permissions, escalation gates, irreversible action blockers.

```
if (memory.trust < T_low || drift > D_high) {
  retrieval.mode = "gated";
  memory.writes = "quarantine";
  agent.tools = "restricted";
}
```

4) Detection → response → recovery workflow

DECISION PIPELINE

Stage	Inputs	Outputs
Detect	trust deltas, drift escalation, signature match, tool misuse	incident candidate, surface attribution, confidence score
Contain	risk score, surface, policy constraints	quarantine vector segment, rotate agent session, gate retrieval
Heal	snapshots, baseline validation, integrity checks	rollback memory, invalidate embeddings, restore baseline
Report	evidence + provenance + actions	structured incident report + SOC export

INCIDENT REPORT STRUCTURE

```
{
  "incidentId": "INC-******",
  "surfaces": ["memory","rag","agent"],
  "detections":
[{"signature":"EmbeddingPoison:GradientFlip","severity":"high"}],
  "actions":
["isolate_vector_segment","rotate_agent_session","rollback_memory"],
  "recommendedControls": [...]
}
```

5) Architecture integration

WHERE DAIFEND SITS

- Between **apps** and **agents** as a runtime policy and telemetry layer.
- Alongside **RAG ingestion** and **retrieval** for scanning + chain validation.
- With **vector databases** for trust scoring, lineage, and rollback operations.
- Exports to **SOC** systems for alerting and incident response coordination.

INTEGRATIONS

- LLMs: OpenAI, Azure OpenAI, AWS Bedrock, Anthropic
- Vector DBs: Pinecone, Weaviate
- Frameworks: LangChain
- Runtime: Kubernetes
- SOC: CrowdStrike, Microsoft Sentinel